

Tuần 29 - Thứ 2 buổi sáng

Numpy cơ bản - Arrays và operations

Thời gian: 09:00-12:00

Mục tiêu bài học

- Hiểu Numpy là gì và tại sao cần dùng
- Tạo và thao tác với numpy arrays
- Thực hiện các phép toán cơ bản với arrays
- Áp dụng kiến thức đã học về Python vào Numpy

1. Numpy là gì?

NumPy (Numerical Python) là thư viện cơ bản cho tính toán khoa học trong Python.

Tại sao cần Numpy?

- **Nhanh hơn:** Numpy nhanh hơn list thông thường của Python
- **Tiện lợi:** Có nhiều hàm toán học sẵn có
- **Tiết kiệm bộ nhớ:** Sử dụng ít bộ nhớ hơn
- **Cơ sở:** Là nền tảng cho các thư viện khác như Matplotlib, Pandas

So sánh List vs Array

```
In [1]: # Cài đặt và import numpy
import numpy as np

# So sánh List Python vs Numpy Array
print("=== So sánh List vs Array ===")

# List Python
python_list = [1, 2, 3, 4, 5]
print(f"Python List: {python_list}")
print(f"Kiểu dữ liệu: {type(python_list)}")

# Numpy Array
numpy_array = np.array([1, 2, 3, 4, 5])
print(f"Numpy Array: {numpy_array}")
print(f"Kiểu dữ liệu: {type(numpy_array)}")
```

```

=== So sánh List vs Array ===
Python List: [1, 2, 3, 4, 5]
Kiểu dữ liệu: <class 'list'>
Numpy Array: [1 2 3 4 5]
Kiểu dữ liệu: <class 'numpy.ndarray'>

```

TODO 1: Làm quen với Numpy

Nhiệm vụ:

1. Tạo một list Python chứa các số từ 1 đến 10
2. Chuyển đổi list đó thành numpy array
3. In ra và so sánh hai kiểu dữ liệu
4. Thử nhân list với 2 và nhân array với 2, quan sát khác biệt

```

In [ ]: # TODO 1: Viết code ở đây
        # Bước 1: Tạo List từ 1 đến 10

        # Bước 2: Chuyển thành numpy array

        # Bước 3: In ra và so sánh

        # Bước 4: Thử nhân với 2

```

2. Tạo Numpy Arrays

2.1 Các cách tạo array cơ bản

```

In [ ]: # Cách 1: Từ List
arr1 = np.array([1, 2, 3, 4, 5])
print(f"Từ list: {arr1}")

# Cách 2: Từ tuple
arr2 = np.array((1, 2, 3, 4, 5))
print(f"Từ tuple: {arr2}")

# Cách 3: Array 2 chiều (như ma trận)
arr3 = np.array([[1, 2, 3], [4, 5, 6]])
print(f"Array 2D: \n{arr3}")

# Cách 4: Tạo array với giá trị đặc biệt
zeros = np.zeros(5) # Array toàn số 0
ones = np.ones(5) # Array toàn số 1
empty = np.empty(5) # Array rỗng

print(f"Array zeros: {zeros}")
print(f"Array ones: {ones}")
print(f"Array empty: {empty}")

```

2.2 Tạo array với range và linspace

```
In [ ]: # Tương tự range() đã học
arr_range = np.arange(0, 10, 2) # từ 0 đến 10, bước nhảy 2
print(f"Array range: {arr_range}")

# Tạo array với số lượng phần tử cố định
arr_linspace = np.linspace(0, 10, 5) # 5 số từ 0 đến 10
print(f"Array linspace: {arr_linspace}")

# Tạo array ngẫu nhiên
arr_random = np.random.random(5) # 5 số ngẫu nhiên từ 0 đến 1
print(f"Array random: {arr_random}")
```

TODO 2: Tạo Arrays

Nhiệm vụ:

1. Tạo array từ list điểm số [8.5, 7.0, 9.5, 6.5, 8.0]
2. Tạo array 2D biểu diễn ma trận 3x3 với các số từ 1 đến 9
3. Tạo array 10 phần tử toàn số 0
4. Tạo array các số chẵn từ 0 đến 20
5. Tạo array 5 số ngẫu nhiên

```
In [ ]: # TODO 2: Viết code ở đây
# Bước 1: Array từ điểm số

# Bước 2: Array 2D 3x3

# Bước 3: Array 10 phần tử toàn số 0

# Bước 4: Array số chẵn từ 0 đến 20

# Bước 5: Array 5 số ngẫu nhiên
```

3. Thuộc tính của Array

3.1 Các thuộc tính quan trọng

```
In [ ]: # Tạo array để demo
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print(f"Array:\n{arr}")
print("\n== Thuộc tính của Array ==")

# Shape: Kích thước (hàng, cột)
print(f"Shape (kích thước): {arr.shape}")

# Size: Tổng số phần tử
print(f"Size (tổng phần tử): {arr.size}")

# Ndim: Số chiều
print(f"Ndim (số chiều): {arr.ndim}")
```

```
# Dtype: Kiểu dữ liệu
print(f"Dtype (kiểu dữ liệu): {arr.dtype}")

# Itemsize: Kích thước mỗi phần tử (bytes)
print(f"Itemsize (kích thước phần tử): {arr.itemsize} bytes")
```

3.2 Truy cập phần tử (Indexing và Slicing)

```
In [ ]: # Array 1D - giống như List đã học
arr_1d = np.array([10, 20, 30, 40, 50])
print(f"Array 1D: {arr_1d}")

# Truy cập phần tử (index bắt đầu từ 0)
print(f"Phần tử đầu tiên: {arr_1d[0]}")
print(f"Phần tử cuối cùng: {arr_1d[-1]}")

# Slicing (cắt mảng)
print(f"3 phần tử đầu: {arr_1d[:3]}")
print(f"Từ index 2 đến cuối: {arr_1d[2:]}")
print(f"Từ index 1 đến 3: {arr_1d[1:4]}")

print("\n=== Array 2D ===")
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(f"Array 2D:\n{arr_2d}")

# Truy cập phần tử [hàng, cột]
print(f"Phần tử hàng 0, cột 1: {arr_2d[0, 1]}")
print(f"Phần tử hàng 2, cột 2: {arr_2d[2, 2]}")

# Lấy cả hàng hoặc cột
print(f"Hàng đầu tiên: {arr_2d[0, :]}")
print(f"Cột đầu tiên: {arr_2d[:, 0]}")
```



TODO 3: Khám phá Array Properties

Nhiệm vụ:

1. Tạo array 2D kích thước 4x5 với các số từ 1 đến 20
2. In ra tất cả thuộc tính: shape, size, ndim, dtype
3. Truy cập phần tử ở hàng 2, cột 3
4. Lấy hàng thứ 2 và cột thứ 4
5. Lấy 3 phần tử đầu của hàng đầu tiên

```
In [ ]: # TODO 3: Viết code ở đây
# Bước 1: Tạo array 4x5

# Bước 2: In thuộc tính

# Bước 3: Truy cập phần tử [2,3]

# Bước 4: Lấy hàng 2 và cột 4
```

Bước 5: Lấy 3 phần tử đầu của hàng đầu

4. Phép toán cơ bản với Arrays

4.1 Phép toán số học

```
In [ ]: # Tạo arrays để demo
arr1 = np.array([1, 2, 3, 4, 5])
arr2 = np.array([10, 20, 30, 40, 50])

print(f"Array 1: {arr1}")
print(f"Array 2: {arr2}")
print("\n=== Phép toán với arrays ===")

# Phép toán với số
print(f"Array 1 + 10: {arr1 + 10}")
print(f"Array 1 * 3: {arr1 * 3}")
print(f"Array 1 / 2: {arr1 / 2}")
print(f"Array 1 ** 2: {arr1 ** 2}")

print("\n=== Phép toán giữa arrays ===")
# Phép toán giữa arrays (element-wise)
print(f"Array 1 + Array 2: {arr1 + arr2}")
print(f"Array 1 * Array 2: {arr1 * arr2}")
print(f"Array 2 - Array 1: {arr2 - arr1}")
print(f"Array 2 / Array 1: {arr2 / arr1}")
```

4.2 Hàm toán học

```
In [ ]: # Tạo array để demo
arr = np.array([1, 4, 9, 16, 25])
print(f"Array gốc: {arr}")

# Hàm căn bậc hai
print(f"Căn bậc hai: {np.sqrt(arr)}")

# Hàm lũy thừa
print(f"Lũy thừa bậc 2: {np.power(arr, 2)}")

# Hàm Lượng giác
angles = np.array([0, 30, 45, 60, 90])
# Chuyển độ sang radian
radians = np.radians(angles)
print(f"Góc (độ): {angles}")
print(f"Sin: {np.sin(radians)}")
print(f"Cos: {np.cos(radians)}")

# Hàm Logarit
arr_log = np.array([1, 2, 4, 8, 16])
print(f"Array: {arr_log}")
```

```
print(f"Log tự nhiên: {np.log(arr_log)}")
print(f"Log cơ số 2: {np.log2(arr_log)}")
```

4.3 Hàm thống kê

```
In [ ]: # Tạo array điểm số
scores = np.array([85, 90, 78, 92, 88, 76, 95, 82, 89, 91])
print(f"Điểm số: {scores}")
print("\n=== Thống kê cơ bản ===")

# Các hàm thống kê
print(f"Tổng: {np.sum(scores)}")
print(f"Trung bình: {np.mean(scores)}")
print(f"Giá trị lớn nhất: {np.max(scores)}")
print(f"Giá trị nhỏ nhất: {np.min(scores)}")
print(f"Độ lệch chuẩn: {np.std(scores)}")
print(f"Phương sai: {np.var(scores)}")

# Tìm vị trí (index)
print(f"Vị trí điểm cao nhất: {np.argmax(scores)}")
print(f"Vị trí điểm thấp nhất: {np.argmin(scores)}")

# Sắp xếp
print(f"Sắp xếp tăng dần: {np.sort(scores)}")
print(f"Sắp xếp giảm dần: {np.sort(scores)[::-1]}")
```



TODO 4: Thực hành phép toán

Nhiệm vụ:

1. Tạo array nhiệt độ trong tuần: [25, 28, 30, 27, 26, 29, 31]
2. Chuyển đổi từ độ C sang độ F (công thức: $F = C * 9/5 + 32$)
3. Tính nhiệt độ trung bình, cao nhất, thấp nhất
4. Tạo array các số từ 1 đến 10, tính tổng bình phương
5. Tạo array góc [0, 30, 60, 90] độ, tính sin và cos

```
In [ ]: # TODO 4: Viết code ở đây
# Bước 1: Array nhiệt độ

# Bước 2: Chuyển C sang F

# Bước 3: Thống kê nhiệt độ

# Bước 4: Tổng bình phương

# Bước 5: Sin và cos
```

5. Thay đổi hình dạng Array

5.1 Reshape - Thay đổi kích thước

```
In [ ]: # Tạo array 1D
arr_1d = np.arange(12) # 0, 1, 2, ..., 11
print(f"Array 1D: {arr_1d}")
print(f"Shape: {arr_1d.shape}")

# Reshape thành 2D
arr_2d = arr_1d.reshape(3, 4) # 3 hàng, 4 cột
print(f"\nArray 2D (3x4):\n{arr_2d}")
print(f"Shape: {arr_2d.shape}")

# Reshape thành 2D khác
arr_2d_v2 = arr_1d.reshape(4, 3) # 4 hàng, 3 cột
print(f"\nArray 2D (4x3):\n{arr_2d_v2}")

# Reshape về 1D
arr_back_1d = arr_2d.reshape(12) # hoặc dùng .flatten()
print(f"\nVề 1D: {arr_back_1d}")

# Dùng -1 để tự động tính
arr_auto = arr_1d.reshape(2, -1) # 2 hàng, tự động tính cột
print(f"\nReshape với -1 (2x?):\n{arr_auto}")
```

5.2 Transpose - Chuyển vị ma trận

```
In [ ]: # Tạo ma trận 2x3
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(f"Ma trận gốc (2x3):\n{matrix}")
print(f"Shape: {matrix.shape}")

# Transpose (chuyển vị)
matrix_T = matrix.T # hoặc np.transpose(matrix)
print(f"\nMa trận chuyển vị (3x2):\n{matrix_T}")
print(f"Shape: {matrix_T.shape}")

# Ví dụ thực tế: Bảng điểm
print("\n=== Ví dụ bảng điểm ===")
scores = np.array([[85, 90, 78], # Học sinh 1
                   [92, 88, 76], # Học sinh 2
                   [95, 82, 89]]) # Học sinh 3
print("Điểm theo học sinh (hàng = học sinh):")
print(scores)

# Chuyển vị để có điểm theo môn
scores_by_subject = scores.T
print("\nĐiểm theo môn học (hàng = môn):")
print(scores_by_subject)

# Tính điểm trung bình từng môn
```

```
avg_by_subject = np.mean(scores_by_subject, axis=1)
print(f"\nĐiểm TB từng môn: {avg_by_subject}")
```

TODO 5: Thay đổi hình dạng

Nhiệm vụ:

1. Tạo array 1D có 24 phần tử (từ 1 đến 24)
2. Reshape thành ma trận 4x6 và 6x4
3. Tạo ma trận 3x4 với số ngẫu nhiên
4. Chuyển vị ma trận đó
5. Tính tổng theo hàng và theo cột của ma trận gốc

```
In [ ]: # TODO 5: Viết code ở đây
# Bước 1: Array 1D có 24 phần tử

# Bước 2: Reshape thành 4x6 và 6x4

# Bước 3: Ma trận 3x4 ngẫu nhiên

# Bước 4: Chuyển vị

# Bước 5: Tổng theo hàng và cột
```

6. Ứng dụng thực tế

6.1 Xử lý điểm số học sinh

```
In [ ]: # Dữ liệu điểm số lớp học
# Mỗi hàng là 1 học sinh, mỗi cột là 1 môn (Toán, Lý, Hóa, Anh, Văn)
class_scores = np.array([
    [8.5, 7.0, 8.0, 9.0, 7.5], # Học sinh 1
    [9.0, 8.5, 7.5, 8.0, 8.5], # Học sinh 2
    [7.5, 6.5, 9.0, 7.0, 8.0], # Học sinh 3
    [8.0, 9.0, 6.5, 8.5, 7.5], # Học sinh 4
    [6.5, 7.5, 8.5, 7.5, 9.0] # Học sinh 5
])

subjects = ['Toán', 'Lý', 'Hóa', 'Anh', 'Văn']
print(f"Bảng điểm lớp ({class_scores.shape[0]} học sinh, {class_scores.shape[1]} môn)")
print(class_scores)

# Tính điểm trung bình từng học sinh
student_avg = np.mean(class_scores, axis=1) # axis=1 tính theo hàng
print(f"\nĐiểm TB từng học sinh: {student_avg}")

# Tính điểm trung bình từng môn
subject_avg = np.mean(class_scores, axis=0) # axis=0 tính theo cột
print(f"Điểm TB từng môn: {subject_avg}")
for i, subject in enumerate(subjects):
    print(f" {subject}: {subject_avg[i]:.2f}")
```



```
# Tìm học sinh giỏi nhất và yếu nhất
best_student = np.argmax(student_avg)
worst_student = np.argmin(student_avg)
print(f"\nHọc sinh giỏi nhất: Học sinh {best_student + 1} (TB: {student_avg[best_student]})")
print(f"Học sinh cần cải thiện: Học sinh {worst_student + 1} (TB: {student_avg[worst_student]})")

# Tìm môn khó nhất và dễ nhất
hardest_subject = np.argmin(subject_avg)
easiest_subject = np.argmax(subject_avg)
print(f"\nMôn khó nhất: {subjects[hardest_subject]} (TB: {subject_avg[hardest_subject]})")
print(f"Môn dễ nhất: {subjects[easiest_subject]} (TB: {subject_avg[easiest_subject]})")
```

6.2 Mô phỏng nhiệt độ hàng ngày

```
In [ ]: # Tạo dữ liệu nhiệt độ trong 1 tháng (30 ngày)
np.random.seed(42) # Để kết quả giống nhau mỗi lần chạy
daily_temp = np.random.normal(28, 3, 30) # TB=28°C, độ lệch=3°C
daily_temp = np.round(daily_temp, 1) # Làm tròn 1 chữ số

print(f"Nhiệt độ 30 ngày: {daily_temp}")
print(f"\n=== Phân tích nhiệt độ ===")
print(f"Nhiệt độ trung bình: {np.mean(daily_temp):.1f}°C")
print(f"Nhiệt độ cao nhất: {np.max(daily_temp):.1f}°C (ngày {np.argmax(daily_temp)})")
print(f"Nhiệt độ thấp nhất: {np.min(daily_temp):.1f}°C (ngày {np.argmin(daily_temp)})")
print(f"Độ lệch chuẩn: {np.std(daily_temp):.1f}°C")

# Đếm số ngày nóng (>30°C) và Lạnh (<25°C)
hot_days = np.sum(daily_temp > 30)
cold_days = np.sum(daily_temp < 25)
normal_days = 30 - hot_days - cold_days

print(f"\n=== Phân loại ngày ===")
print(f"Ngày nóng (>30°C): {hot_days} ngày")
print(f"Ngày lạnh (<25°C): {cold_days} ngày")
print(f"Ngày bình thường: {normal_days} ngày")

# Tìm các ngày có nhiệt độ bất thường
mean_temp = np.mean(daily_temp)
std_temp = np.std(daily_temp)
abnormal_days = np.where(np.abs(daily_temp - mean_temp) > 2 * std_temp)[0]

if len(abnormal_days) > 0:
    print(f"\n=== Ngày có nhiệt độ bất thường ===")
    for day in abnormal_days:
        print(f"Ngày {day + 1}: {daily_temp[day]}°C")
else:
    print(f"\nKhông có ngày nào có nhiệt độ bất thường")
```



TODO 6: Dự án mini - Phân tích doanh thu

Tình huống: Bạn là nhân viên phân tích của một cửa hàng. Hãy phân tích doanh thu bán hàng.

Nhiệm vụ:

1. Tạo dữ liệu doanh thu 7 ngày trong tuần cho 4 tuần (shape: 4x7)
2. Tính tổng doanh thu từng ngày trong tuần
3. Tính tổng doanh thu từng tuần
4. Tìm ngày có doanh thu cao nhất và thấp nhất
5. Tính doanh thu trung bình và độ lệch chuẩn
6. So sánh doanh thu cuối tuần (T7, CN) với ngày thường

```
In [ ]: # TODO 6: Dự án mini - Phân tích doanh thu
# Tạo dữ liệu doanh thu (triệu đồng)
# Gợi ý: dùng np.random.randint(50, 200, (4, 7)) cho doanh thu từ 50-200 triệu

# Bước 1: Tạo dữ liệu doanh thu 4 tuần x 7 ngày
revenue = # Viết code ở đây

# In dữ liệu
days = ['T2', 'T3', 'T4', 'T5', 'T6', 'T7', 'CN']
print("Doanh thu 4 tuần (triệu đồng):")
print(revenue)

# Bước 2: Tổng doanh thu từng ngày trong tuần

# Bước 3: Tổng doanh thu từng tuần

# Bước 4: Ngày có doanh thu cao nhất và thấp nhất

# Bước 5: Doanh thu trung bình và độ Lệch chuẩn

# Bước 6: So sánh cuối tuần vs ngày thường
# weekend_revenue = # T7 và CN (cột 5, 6)
# weekday_revenue = # T2-T6 (cột 0-4)
```

7. Tổng kết và kiến thức mở rộng

7.1 Những điều đã học

Hôm nay chúng ta đã học:

1. **Numpy là gì:** Thư viện tính toán nhanh cho Python
2. **Tạo arrays:** Từ list, zeros, ones, arange, linspace, random
3. **Thuộc tính arrays:** shape, size, ndim, dtype
4. **Truy cập dữ liệu:** indexing, slicing cho 1D và 2D
5. **Phép toán:** +, -, *, /, **, với số và giữa arrays
6. **Hàm toán học:** sqrt, sin, cos, log, các hàm thống kê
7. **Thay đổi hình dạng:** reshape, transpose
8. **Ứng dụng thực tế:** Xử lý điểm số, phân tích dữ liệu

Kiến thức từ các tuần trước đã sử dụng:

- Variables và data types (W26)
- Lists và indexing (W27)
- Functions và methods (W28)
- For loops và conditions (W26)

7.2 So sánh List vs Numpy Array

```
In [ ]: # Demo cuối: So sánh hiệu suất
import time

# Tạo dữ liệu lớn
size = 100000
python_list = list(range(size))
numpy_array = np.arange(size)

print(f"So sánh hiệu suất với {size} phần tử:")

# Test với Python List
start_time = time.time()
result_list = [x * 2 for x in python_list]
list_time = time.time() - start_time

# Test với Numpy array
start_time = time.time()
result_array = numpy_array * 2
array_time = time.time() - start_time

print(f"Python List: {list_time:.4f} giây")
print(f"Numpy Array: {array_time:.4f} giây")
print(f"Numpy nhanh hơn: {list_time/array_time:.1f} lần")

# So sánh bộ nhớ
import sys
list_memory = sys.getsizeof(python_list[:1000])
array_memory = numpy_array[:1000].nbytes
print(f"\nBộ nhớ (1000 phần tử):")
print(f"Python List: {list_memory} bytes")
print(f"Numpy Array: {array_memory} bytes")
print(f"Numpy tiết kiệm: {list_memory/array_memory:.1f} lần")
```

7.3 Chuẩn bị cho buổi chiều

Buổi chiều (14:00-17:00) sẽ học:

- Matrix calculation (tính toán ma trận)
- Array manipulation (thao tác nâng cao với arrays)
- Image as arrays intro (giới thiệu ảnh như arrays)

Bài tập về nhà:

1. Ôn lại tất cả các TODO đã làm

2. Thử tạo một ứng dụng nhỏ phân tích điểm số của riêng bạn
3. Tìm hiểu thêm về numpy functions: np.where, np.concatenate

Câu hỏi thảo luận:

- Khi nào nên dùng List, khi nào nên dùng Numpy Array?
- Numpy có thể ứng dụng vào những lĩnh vực nào khác?
- Làm thế nào để xử lý dữ liệu lớn hiệu quả hơn?

Bài tập cuối buổi

Thử thách: Tạo một chương trình phân tích kết quả thi của lớp học

Yêu cầu:

1. Tạo dữ liệu điểm thi cho 20 học sinh, 5 môn học
2. Tính điểm trung bình từng học sinh và từng môn
3. Xếp hạng học sinh theo điểm TB
4. Tìm ra 3 học sinh giỏi nhất và 3 học sinh cần hỗ trợ
5. Phân tích môn học nào khó nhất, dễ nhất
6. Đếm số học sinh đạt, không đạt (≥ 5.0 điểm TB)

Sử dụng tất cả kiến thức đã học trong buổi này!

```
In [ ]: # Bài tập cuối buổi - Viết code ở đây
        # Tạo chương trình phân tích kết quả thi

        # Gợi ý structure:
        # 1. Tạo dữ liệu
        # 2. Tính toán cơ bản
        # 3. Xếp hạng và phân loại
        # 4. Phân tích và báo cáo

        print("=== HỆ THỐNG PHÂN TÍCH KẾT QUẢ THI ===")
        # Viết code của bạn ở đây...
```